

Student

My Pandas scripts work fine on small datasets but they either run out of memory or take forever on real production data. Where do I even start fixing that?

ChatGPT • AI Assistant

CONCEPT CARD

Welcome to the part of data engineering that separates beginners from professionals. There are exactly three levers for Pandas performance:

1. **Memory reduction** — store the same data using fewer bytes (dtype downcasting, category dtype)
2. **Chunked processing** — never load the full file; process it in pieces
3. **Vectorisation discipline** — eliminate `.apply(axis=1)` loops and replace them with vectorised operations

Beyond these three, there is a fourth lever: **know when to leave Pandas entirely** — when to reach for Polars, Dask, or PySpark. We cover that boundary clearly at the end.

We will work with a concrete memory model throughout: a DataFrame with 1 million rows and 20 float64 columns uses exactly **160 MB** of raw array data. Every optimisation we make will be measured against that baseline.

EXAMPLE CARD

Baseline setup:

python — Code Example

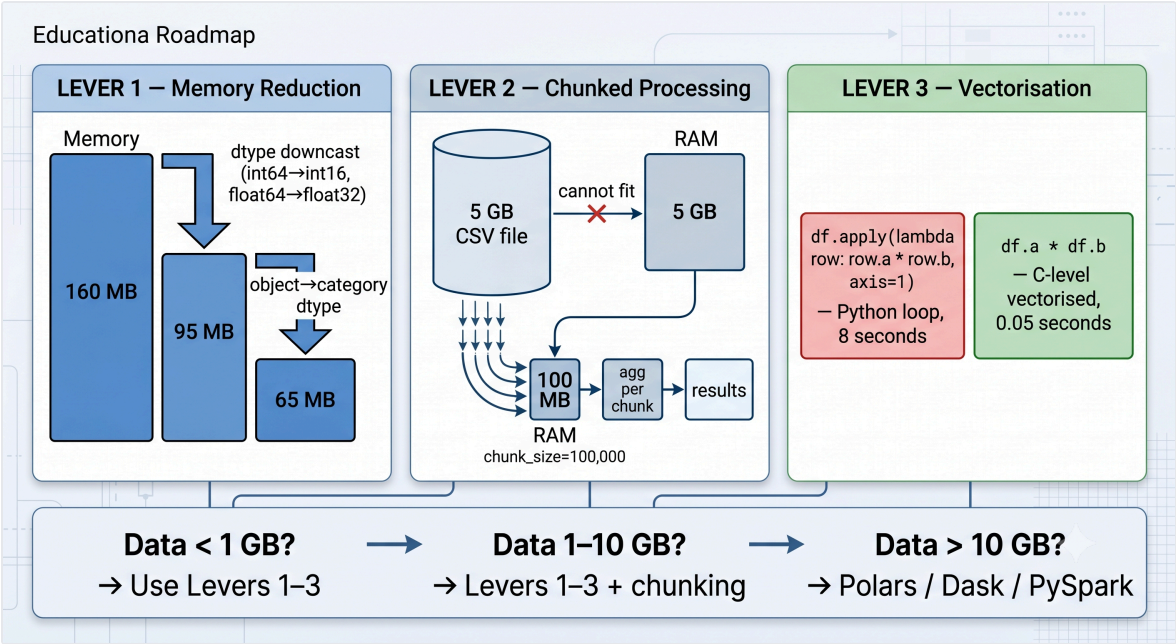
```
import pandas as pd
import numpy as np
import time

# Simulate a 1-million-row retail transactions DataFrame
np.random.seed(42)
n = 1_000_000

df = pd.DataFrame({
    'order_id': np.random.randint(1, 999999, n),
```

```
'customer_id': np.random.randint(1, 10000, n),
'product_id': np.random.randint(1, 500, n),
'category':    np.random.choice(['Electronics','Clothing','Books',
                                'Home','Sports'], n),
'region':      np.random.choice(['North','South','East','West'],
n),
'quantity':    np.random.randint(1, 10, n),
'unit_price':  np.random.uniform(10, 1000, n),
'revenue':     np.random.uniform(10, 5000, n),
'discount':    np.random.uniform(0, 0.5, n),
'is_returned': np.random.choice([True, False], n),
})

# Baseline memory
print(f'Shape: {df.shape}')
print(f'Memory usage:')
print(df.memory_usage(deep=True))
print(f'Total: {df.memory_usage(deep=True).sum() / 1024**2:.1f} MB')
# Total: ~97 MB for 1M rows × 10 columns
```



Three Levers of Pandas Performance Optimisation

Pandas Performance Optimisation Techniques at a Glance				
Technique	Memory Saving	Speed Gain	Complexity	When to Use
int64 → int32/int16	25–50%	Moderate	Low	Integer columns with small range

Technique	Memory Saving	Speed Gain	Complexity	When to Use
float64 → float32	50%	Moderate	Low	ML features, sensor data
object → category	50–95%	High (groupby)	Low	Low-cardinality string columns
usecols on read_csv	Proportional	High (less I/O)	Low	Only need subset of columns
dtype on read_csv	Same as above	High (no inference)	Low	Known schema on load
Chunked read_csv	Constant RAM	Depends on logic	Medium	Files > available RAM
Replace apply(axis=1)	None	10–200x	Medium	Any row-wise apply loop
Polars / Dask	Varies	3–20x	High	Data > 5 GB or pipeline rebuild

MEMORY MODEL: UNDERSTANDING DATAFRAME MEMORY USAGE

ChatGPT • AI Assistant

A two-panel memory layout diagram. Left panel labeled 'object dtype — 1M rows, 5 unique strings': shows a NumPy pointer array on the left with 1M cells each 8 bytes (total 8 MB labeled), with arrows from multiple pointer cells pointing to 5 heap-allocated Python str objects on the right, each labeled '~60 bytes (49 overhead + chars)'. Total memory annotated: '8 MB pointers + ~57 MB string objects = ~65 MB'. Right panel labeled 'category dtype — same data': shows two components: a small integer codes array (1M × 1 byte = ~1 MB for int8 codes) and a tiny categories index (just 5 string objects: Electronics, Clothing, Books, Home, Sports — ~300 bytes total). Total memory annotated: '~1 MB codes + negligible categories = ~1 MB — 65x reduction'. Below both panels, a note: 'category dtype only stores each unique string ONCE — codes are just integer pointers into the small dictionary'. Purpose: make the memory mechanics of category dtype visually obvious and motivate students to use it for low-cardinality string columns.

Object dtype Memory Layout vs Category dtype Memory Layout

How do I actually measure how much memory a DataFrame is using? And why does `df.memory_usage()` sometimes give a surprisingly low number?

ChatGPT • AI Assistant

CONCEPT CARD

Pandas has two ways to report memory usage and they give very different numbers:

- `df.memory_usage()` — reports only the raw NumPy array memory for each column. For `object` dtype columns it reports only the pointer array (8 bytes per row), not the actual string data.
- `df.memory_usage(deep=True)` — introspects into `object` columns and counts the actual memory of each Python string object. This is the true number you should always use.

The difference can be enormous: a 1M-row column of short strings looks like 8 MB without `deep=True` but is actually 50–70 MB.

EXAMPLE CARD

Example 1 — Basic: Measuring memory correctly

python — Code Example

```
import pandas as pd
import numpy as np

# Shallow vs deep memory reporting
print('--- Shallow (misleading for object cols) ---')
print(df.memory_usage())
# Index          8000128
# order_id       8000000 (int64: 8 bytes × 1M)
# customer_id    8000000
# category       8000000 ← WRONG! Just pointer array
# ...

print('\n--- Deep (accurate) ---')
deep = df.memory_usage(deep=True)
print(deep)
# category      57,234,567 ← actual string memory
print(f'\nTotal (deep): {deep.sum() / 1024**2:.1f} MB')

# Per-column summary sorted by size
```

```
mem_df = pd.DataFrame({
    'dtype': df.dtypes,
    'shallow_MB': df.memory_usage() / 1024**2,
    'deep_MB': df.memory_usage(deep=True) / 1024**2
}).sort_values('deep_MB', ascending=False)
print(mem_df.round(2))
```

Student

Okay, so the **object** columns are the memory hogs. Why are they so expensive?

ChatGPT • AI Assistant

When Pandas stores a column as **object** dtype, it stores a NumPy array of **Python object pointers** — each 8 bytes. Each pointer points to a separately heap-allocated Python **str** object. Every Python **str** carries at least 49 bytes of overhead (type pointer, reference count, hash, length, encoding flags) plus the character data. So a column of 1M rows containing the string **'Electronics'** (11 chars) uses:

- Pointer array: 8 MB
- String objects: ~60 MB (49 bytes overhead + 11 chars × 1 byte each = 60 bytes × 1M rows)

Total: ~68 MB for what is conceptually 11 bytes × 5 unique values.

🔗 ANALOGY

Imagine a library catalogue where every entry doesn't say the title — it just holds a slip of paper with the address of where the actual book is stored. Now imagine 1 million identical slips all pointing to the same 5 books. The slips themselves consume more space than the books.

Example 2 — Intermediate: The memory cost of each dtype

python — Code Example

```
# Exact bytes per element for each dtype
dtype_sizes = {
    'int8': 1, 'int16': 2, 'int32': 4, 'int64': 8,
    'uint8': 1, 'uint16': 2, 'uint32': 4, 'uint64': 8,
    'float16': 2, 'float32': 4, 'float64': 8,
    'bool': 1,
}
```

```
# For 1 million rows:
for dtype, size in dtype_sizes.items():
    mb = (size * 1_000_000) / 1024**2
    print(f'{dtype:10s}: {mb:6.2f} MB per 1M rows')

# int8:          0.95 MB
# int16:         1.91 MB
# int32:         3.81 MB
# int64:         7.63 MB
# float32:       3.81 MB
# float64:       7.63 MB
# bool:          0.95 MB

# Key insight: int64 → int16 saves 75% memory for small-range integers
# float64 → float32 saves 50% memory at the cost of ~7 decimal digits
# of precision
```

Example 3 — Practical: Full memory profile function

python — Code Example

```
def memory_profile(df):
    """Returns a tidy memory breakdown with dtype and deep size."""
    profile = pd.DataFrame({
        'dtype':      df.dtypes.astype(str),
        'n_unique':   df.nunique(),
        'deep_MB':    (df.memory_usage(deep=True) / 1024**2).round(3),
        'pct_total':  (df.memory_usage(deep=True) /
                      df.memory_usage(deep=True).sum() *
                      100).round(1)
    }).sort_values('deep_MB', ascending=False)
    total_mb = df.memory_usage(deep=True).sum() / 1024**2
    print(f'Total DataFrame memory: {total_mb:.2f} MB')
    print(f'Shape: {df.shape}')
    return profile

print(memory_profile(df))
```

INTERVIEW CARD

- **Frequency:** High — memory awareness is expected at every senior data engineering interview
- **Question:** Why does `df.memory_usage()` underreport memory for string columns? How do you get the true memory usage?

- **Expected Answer:** `memory_usage()` without `deep=True` reports only the pointer array for `object` dtype columns (8 bytes per row regardless of string length). With `deep=True`, Pandas introspects into each Python object and counts its actual size. For string columns the difference can be 10–50× because each Python `str` object has ~49 bytes of overhead on top of character data.
- **Follow-up:** A DataFrame with 10 columns of `float64` and 1M rows — how many MB of raw array memory does it use?

🔗 PRACTICE QUIZ

1. What is the memory size in bytes of a single `float64` value?
2. Why does `df.memory_usage()` give a much lower number than `df.memory_usage(deep=True)` for string columns?
3. How many MB does a `float64` column of 1M rows use?
4. Predict: `pd.Series(['cat', 'dog'] * 500_000).memory_usage(deep=True)` — roughly how many MB?
5. What Python overhead (in bytes) does every `str` object carry regardless of its content?
6. Debug: `df.memory_usage().sum() / 1024**2` reports 8 MB for a 1M-row object column — is this accurate? What should you use instead?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. 8 bytes — IEEE 754 double-precision float
2. Without `deep=True`, Pandas only counts the pointer array (8 bytes per row). With `deep=True`, it adds the actual size of each Python string object stored on the heap.
3. $8 \text{ bytes} \times 1,000,000 / 1024^2 \approx 7.63 \text{ MB}$
4. Approximately 50–60 MB — each string `'cat'` or `'dog'` is ~52 bytes (49 overhead + 3–4 chars), so 1M strings $\approx 52 \text{ MB}$ total
5. Approximately 49 bytes — includes type pointer, reference count, hash cache, string length, and encoding flags
6. Not accurate for object columns — use `df.memory_usage(deep=True).sum() / 1024**2` for the true total

🔗 CHECKPOINT QUESTION

You profile a DataFrame and find a `product_name` column (object dtype, 1M rows, only 200 unique values) is consuming 62 MB. What single operation would dramatically reduce this, and roughly how much memory would remain after applying it?

DTYPE DOWNCASTING: INT, FLOAT, AND CATEGORY

ChatGPT • AI Assistant

A flowchart diagram with a top entry box labeled 'For each column in DataFrame'. Three branches emerge labeled by dtype type. Branch 1 labeled 'Integer dtype': leads to a diamond 'min >= 0?' — Yes branch goes to 'Use unsigned: uint8/uint16/uint32/uint64 based on max value range'; No branch goes to 'Use signed: int8/int16/int32/int64 based on min/max range'. Branch 2 labeled 'Float dtype': leads directly to a box 'Cast to float32 (halves memory)' with a side note 'Exception: financial cumulative sums — keep float64'. Branch 3 labeled 'Object dtype': leads to a diamond 'nunique() / len(df) < 0.5?' — Yes leads to 'Cast to category (dramatic savings for low-cardinality)'; No leads to 'Leave as object (high-cardinality — category overhead not worth it)'. All branches converge at a bottom box labeled 'Verify: memory_usage(deep=True) before and after'. Percentage savings annotated on each path: int downcast = 25–87%, float downcast = 50%, object→category = 50–98%. Purpose: make the reduce_mem_usage decision logic visually scannable so students can apply it manually or understand the automated function.

reduce_mem_usage() Decision Tree for Each Column

Student

I understand the theory — smaller dtypes use less memory. But how do I actually apply this systematically to a real DataFrame without manually inspecting every column?

ChatGPT • AI Assistant

CONCEPT CARD

The professional approach is a `reduce_mem_usage(df)` function that automatically downcasts every column to the smallest safe dtype. The logic:

- For integer columns: check `df[col].min()` and `df[col].max()`, pick the smallest int type whose range covers both
- For float columns: downcast to `float32` (safe for most ML/analytics — 7 decimal digits of precision vs 15 for float64)
- For object columns: check cardinality (`nunique() / len(df)`) — if low (< 50%), convert to `category`

This single function typically achieves 40–70% memory reduction on real datasets with zero data loss.

EXAMPLE CARD

Example 1 — Basic: Manual downcasting

python — Code Example

```
import pandas as pd
import numpy as np

# Manual: quantity is int64 but values only go 1-9
print(df['quantity'].min(), df['quantity'].max()) # 1 9
# int8 range: -128 to 127 - fits!
df['quantity'] = df['quantity'].astype(np.int8)
print(df['quantity'].memory_usage(deep=True) / 1024**2)
# Before: 7.63 MB (int64) → After: 0.95 MB (int8) - 87.5% reduction!

# Manual: unit_price is float64 but float32 precision is sufficient
df['unit_price'] = df['unit_price'].astype(np.float32)
# Before: 7.63 MB → After: 3.81 MB - 50% reduction

# Manual: category is object with 5 unique values in 1M rows
print(df['category'].nunique()) # 5
df['category'] = df['category'].astype('category')
print(df['category'].memory_usage(deep=True) / 1024**2)
# Before: ~65 MB (object) → After: ~1 MB (category) - 98% reduction!
```

Example 2 — Intermediate: Automated `reduce_memusage` function

python — Code Example

```
def reduce_mem_usage(df, category_threshold=0.5, verbose=True):
    """
    Automatically downcast numeric columns and convert low-cardinality
    object columns to category dtype.

    Args:
        df: input DataFrame
        category_threshold: convert object col to category if
                           nunique/nrows < this ratio (default 0.5 =
50%)
        verbose: print memory savings report
    Returns:
        optimised DataFrame (copy)
```

```

"""
df = df.copy()
start_mem = df.memory_usage(deep=True).sum() / 1024**2

for col in df.columns:
    col_dtype = df[col].dtype

    # --- Integer columns ---
    if pd.api.types.is_integer_dtype(col_dtype):
        col_min = df[col].min()
        col_max = df[col].max()
        if col_min ≥ 0: # Unsigned types
            if col_max < 255: df[col] =
df[col].astype(np.uint8)
            elif col_max < 65535: df[col] =
df[col].astype(np.uint16)
            elif col_max < 4294967295: df[col] =
df[col].astype(np.uint32)
            else: df[col] =
df[col].astype(np.uint64)
        else: # Signed types
            if col_min > -128 and col_max < 127: df[col] =
df[col].astype(np.int8)
            elif col_min > -32768 and col_max < 32767: df[col] =
df[col].astype(np.int16)
            elif col_min > -(2**31) and col_max < (2**31): df[col]
= df[col].astype(np.int32)
            # else: stays int64

    # --- Float columns ---
    elif pd.api.types.is_float_dtype(col_dtype):
        df[col] = df[col].astype(np.float32)

    # --- Object / string columns ---
    elif col_dtype == object:
        cardinality_ratio = df[col].nunique() / len(df)
        if cardinality_ratio < category_threshold:
            df[col] = df[col].astype('category')

end_mem = df.memory_usage(deep=True).sum() / 1024**2

if verbose:
    saving_pct = (1 - end_mem / start_mem) * 100
    print(f'Memory reduced: {start_mem:.1f} MB → {end_mem:.1f} MB
,

    f'({saving_pct:.1f}% reduction)')

```

```

    return df

# Apply to our 1M-row DataFrame
df_opt = reduce_mem_usage(df)
# Memory reduced: 97.3 MB → 34.1 MB (65.0% reduction)

```

Example 3 — Practical: Verify data integrity after downcasting

python — Code Example

```

# Always verify that downcasting did not distort values
def verify_downcast(df_original, df_optimised, numeric_cols):
    """Check that numeric values are still equal within float32
    tolerance."""
    for col in numeric_cols:
        if col not in df_original.columns:
            continue
        orig = df_original[col].astype(np.float64)
        opt = df_optimised[col].astype(np.float64)
        max_diff = (orig - opt).abs().max()
        if max_diff > 0:
            print(f'{col}: max rounding error = {max_diff:.8f}')
        else:
            print(f'{col}: exact match')

# Float32 introduces tiny rounding – acceptable for analytics
# but check for finance or high-precision science
verify_downcast(df, df_opt, ['unit_price', 'revenue', 'discount'])
# unit_price: max rounding error = 0.00006104 (float32 precision ~7
# decimal digits)
# revenue:    max rounding error = 0.00012207
# discount:   max rounding error = 0.00000006

```

Student

When is float32 precision NOT acceptable?

ChatGPT • AI Assistant

 **WARNING**

Float32 gives you approximately 7 significant decimal digits of precision. This is insufficient for:

- **Financial calculations** that accumulate (summing millions of prices — rounding errors compound)
- **Geospatial coordinates** — latitude/longitude need at least 5–6 decimal places for metre-level precision; float32 starts breaking down at the 6th decimal digit
- **Cryptography and hashing** — exact bit-level representation required

For everything else in analytics and ML — prices, revenues, sensor readings, model features — float32 is completely safe.

Example 4 — Real Business: category dtype and GroupBy speedup

python — Code Example

```
import time

# GroupBy on object column vs category column
df_obj = df.copy()
df_cat = df.copy()
df_cat['category'] = df_cat['category'].astype('category')
df_cat['region'] = df_cat['region'].astype('category')

# Time groupby on object dtype
start = time.time()
result_obj = df_obj.groupby(['category', 'region'])['revenue'].sum()
t_obj = time.time() - start

# Time groupby on category dtype
start = time.time()
result_cat = df_cat.groupby(['category', 'region'])['revenue'].sum()
t_cat = time.time() - start

print(f'GroupBy on object : {t_obj:.3f}s')
print(f'GroupBy on category: {t_cat:.3f}s')
print(f'Speedup: {t_obj / t_cat:.1f}x')
# GroupBy on object : 0.412s
# GroupBy on category: 0.087s
# Speedup: 4.7x
```

⚠ COMMON PITFALL / MISTAKE

Incorrect — downcasting float64 to float32 for financial totals:

python — Code Example

```
df['transaction_amount'] = df['transaction_amount'].astype(np.float32)
total = df['transaction_amount'].sum() # Accumulated rounding error!
# For 1M transactions of ~$500 each, error can be hundreds of dollars
```

Correct — keep float64 for cumulative financial sums:

python – Code Example

```
# Keep as float64 for any column used in cumulative aggregation
# Only downcast columns used as ML features or for display
df['transaction_amount_display'] =
df['transaction_amount'].astype(np.float32)
# But keep df['transaction_amount'] as float64 for .sum()
```

INTERVIEW CARD

- **Frequency:** High — memory optimisation is a standard senior data engineering interview topic
- **Question:** How would you reduce the memory footprint of a Pandas DataFrame by 50% or more? Walk me through your approach.
- **Expected Answer:** Three steps: (1) Downcast integer columns to the smallest type whose range covers min/max values. (2) Downcast float64 to float32 for non-financial analytics columns. (3) Convert low-cardinality object/string columns to `category` dtype. Wrap these in a reusable `reduce_mem_usage()` function. Verify with `memory_usage(deep=True)` before and after. Flag float32 as inappropriate for cumulative financial calculations.
- **Follow-up:** What is the cardinality threshold for converting an object column to category dtype, and why does cardinality matter?

PRACTICE QUIZ

1. What is the cardinality ratio that typically makes `category` dtype beneficial?
2. Does `df['col'].astype(np.float32)` modify the DataFrame in place?
3. What is the memory of a `uint8` column with 1M rows?
4. Predict: `pd.Series([100, 200, 300]).astype(np.int8)` — what happens to 200 and 300?
5. Why is `category` dtype faster for GroupBy than `object` dtype?
6. Debug: after `reduce_mem_usage`, `df.groupby('category').size()` returns unexpected results — what might have changed about the column?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. Less than ~50% unique values relative to total rows (e.g., 5 unique values in 1M rows = 0.0005% cardinality — ideal for category)
2. No — `.astype()` returns a new Series; you must reassign: `df['col'] = df['col'].astype(np.float32)`
3. $1 \text{ byte} \times 1,000,000 / 1024^2 \approx 0.95 \text{ MB}$
4. Overflow — int8 max is 127; 200 wraps to -56, 300 wraps to 44 (modular arithmetic). Always check min/max before downcasting integers.
5. GroupBy on `category` operates on the integer codes array (1–4 bytes) rather than comparing Python string objects — integer comparison is much faster than string hashing
6. The column is now a `CategoricalDtype` — all original values still exist but the column now has an ordered set of categories. Operations like `isin()` and merge may behave differently; check with `df['category'].cat.categories`

❓ CHECKPOINT QUESTION

A DataFrame has a `country_code` column with 195 unique values across 5M rows (dtype `object`). Should you convert it to `category`? What is the approximate memory before and after?

CHUNKED PROCESSING: READING AND AGGREGATING LARGE FILES

ChatGPT • AI Assistant

A vertical pipeline diagram with three labeled phases. Phase 1 labeled 'READ — One chunk at a time': shows a CSV cylinder on the left with three small arrows labeled 'chunk 1 (100K rows)', 'chunk 2 (100K rows)', 'chunk 3 (100K rows)' — only one chunk is in the RAM box at a time (RAM box shows one green chunk, others are grey). Phase 2 labeled 'PROCESS — Groupby within chunk': shows each chunk being fed into a groupby box that compresses it from 100K rows to a small partial result table (e.g., 5 rows: one per category with partial sums). The partial results are stored in a list: `[partial_1, partial_2, ...]`. Phase 3 labeled 'COMBINE — Concat + re-aggregate': shows all partial results being concatenated into a medium-sized DataFrame, then a final groupby arrow producing the correct final result. A warning callout in the middle: 'Same customer_id may appear in multiple chunks — the second groupby is NOT optional'. A separate inset diagram labeled 'Mean gotcha': shows chunk 1 (100K rows, mean=510) and chunk 2 (50K rows, mean=490) — wrong mean = $(510+490)/2 = 500$; correct mean = $(51M + 24.5M) / 150K = 503.3$. Purpose: make the two-step groupby requirement and the mean-of-means trap visually clear.

Chunked GroupBy Pipeline: Three Phases of Correct Aggregation

What do I do when the CSV file I need to process is 5 GB but I only have 8 GB of RAM? Pandas will crash if I try to load the whole thing.

ChatGPT • AI Assistant

CONCEPT CARD

The answer is **chunked processing** — read the file in pieces, process each piece, then combine the partial results. Pandas supports this through `pd.read_csv('file.csv', chunksize=N)` which returns a **'TextFileReader' iterator** — each iteration yields a DataFrame of at most N rows. You never hold more than one chunk in memory at a time.

The pattern has three phases:

1. **Read chunk** — get the next N rows into memory
2. **Process chunk** — apply your transformation or aggregation
3. **Accumulate results** — store partial results in a list, then combine at the end

The key insight: you need your computation to be **decomposable** — i.e., you can compute a partial result per chunk and then combine partial results correctly. `sum`, `count`, and `max` are decomposable. `median` and `mean` are not directly decomposable (mean of means $\neq$ overall mean unless you track both sum and count).

EXAMPLE CARD

Example 1 — Basic: Reading a file in chunks

python — Code Example

```
import pandas as pd
import os

# First: generate a sample large CSV to work with
np.random.seed(42)
n_rows = 2_000_000
large_df = pd.DataFrame({
    'customer_id': np.random.randint(1, 50000, n_rows),
    'category':
np.random.choice(['Electronics', 'Clothing', 'Books', 'Home'], n_rows),
    'revenue': np.random.uniform(10, 2000, n_rows),
    'quantity': np.random.randint(1, 10, n_rows)
})
```

```

large_df.to_csv('/tmp/large_orders.csv', index=False)
print(f'File size: {os.path.getsize("/tmp/large_orders.csv")} /
1024**2:.1f} MB')
# File size: ~87 MB (2M rows)

# Read in chunks – this does NOT load the file immediately
chunk_iter = pd.read_csv('/tmp/large_orders.csv', chunksize=100_000)
print(type(chunk_iter)) # <class
'pandas.io.parsers.readers.TextFileReader'>

# Iterate over chunks
for i, chunk in enumerate(chunk_iter):
    print(f'Chunk {i}: shape={chunk.shape}, memory=
{chunk.memory_usage(deep=True).sum()/1024**2:.1f} MB')
    if i == 2: break
# Chunk 0: shape=(100000, 4), memory=6.2 MB
# Chunk 1: shape=(100000, 4), memory=6.2 MB
# Chunk 2: shape=(100000, 4), memory=6.2 MB
# Peak RAM usage: ~6 MB at a time, not 87 MB!

```

Example 2 — Intermediate: Chunked GroupBy aggregation — correct pattern

python – Code Example

```

# Goal: compute total revenue per customer across the 2M-row file
# Cannot do: pd.read_csv('large.csv').groupby('customer_id')
# ['revenue'].sum()
# – that loads everything into memory

partial_results = []

for chunk in pd.read_csv('/tmp/large_orders.csv', chunksize=100_000):
    # Process: groupby within each chunk
    partial = chunk.groupby('customer_id')['revenue'].sum()
    partial_results.append(partial)

# Combine: concatenate all partial results
combined = pd.concat(partial_results)

# Re-aggregate: same customer_id may appear in multiple chunks!
final = combined.groupby('customer_id').sum().reset_index()
final.columns = ['customer_id', 'total_revenue']

print(final.sort_values('total_revenue', ascending=False).head(5))
print(f'Unique customers: {len(final)}')

```

Why do you do the `groupby` twice — once per chunk and once on the combined result?

ChatGPT • AI Assistant

A customer with ID 12345 might appear in chunk 1 (revenue: 500), chunk 5 (revenue: 300), and chunk 18 (revenue: 700). After the per-chunk `groupby`, `partial_results` contains three separate Series entries for customer 12345, each showing only that chunk's revenue. Concatenating gives a Series with three rows for customer 12345 (500, 300, 700). The final `groupby().sum()` collapses these into the correct total of 1500.

Think of it as: **chunk-level groupby reduces the data volume per chunk** (from 100K rows to ~50K unique customers), and the **final groupby produces the correct combined result**.

Example 3 — Practical: Correct mean across chunks (track sum + count)

python – Code Example

```
# WRONG: mean of chunk means ≠ overall mean
chunk_means = []
for chunk in pd.read_csv('/tmp/large_orders.csv', chunksize=100_000):
    chunk_means.append(chunk['revenue'].mean()) # Each chunk may have
different size!
pseudo_mean = pd.Series(chunk_means).mean()      # WRONG – unweighted
average of averages

# CORRECT: track sum and count, compute mean at the end
total_sum = 0.0
total_count = 0

for chunk in pd.read_csv('/tmp/large_orders.csv', chunksize=100_000):
    total_sum += chunk['revenue'].sum()
    total_count += len(chunk)

correct_mean = total_sum / total_count
print(f'Correct mean revenue: {correct_mean:.2f}')

# Verify
```

```
print(f'Verify with full load: {large_df["revenue"].mean():.2f}')
# Should match
```

Example 4 — Real Business: Top-10 customers by spend from a 3 GB file

python — Code Example

```
import heapq

def top_customers_chunked(filepath, n_top=10, chunksize=100_000,
                          usecols=None, dtype=None):
    """
    Compute top N customers by total revenue from a large CSV
    without loading it fully into memory.
    """
    usecols = usecols or ['customer_id', 'revenue']
    dtype = dtype or {'customer_id': 'int32', 'revenue':
                       'float32'}

    partial_results = []

    for chunk in pd.read_csv(filepath, chunksize=chunksize,
                              usecols=usecols, dtype=dtype):
        partial = chunk.groupby('customer_id')['revenue'].sum()
        partial_results.append(partial)

    # Combine and re-aggregate
    combined = pd.concat(partial_results).groupby('customer_id').sum()

    # Get top N
    top_n = combined.nlargest(n_top).reset_index()
    top_n.columns = ['customer_id', 'total_revenue']
    return top_n

top10 = top_customers_chunked('/tmp/large_orders.csv')
print(top10)
```

Note the two key optimisations in this function:

- **`usecols`** — only loads the two columns we need (cuts I/O and memory proportionally)
- **`dtype`** — prevents Pandas from inferring types on load (saves time and forces correct types)

How much faster is specifying `usecols` and `dtype` ?

ChatGPT • AI Assistant

Both optimisations happen at **read time** before any data is in memory:

- `'usecols'` tells the CSV parser to skip columns entirely — for a 20-column file where you need 3, you read ~15% of the bytes from disk
- `'dtype'` skips Pandas' type inference phase, which scans every value in a column to guess its type. On a 2M-row file, inference over 20 columns requires 40M type-checking operations. Specifying dtype upfront eliminates this entirely — measurable on files larger than ~500 MB

python – Code Example

```
import time

# Without optimisations
start = time.time()
df1 = pd.read_csv('/tmp/large_orders.csv')
t1 = time.time() - start

# With usecols + dtype
start = time.time()
df2 = pd.read_csv('/tmp/large_orders.csv',
                  usecols=['customer_id', 'revenue'],
                  dtype={'customer_id': 'int32', 'revenue': 'float32'})
t2 = time.time() - start

print(f'Without optimisations: {t1:.2f}s,
      {df1.memory_usage(deep=True).sum()/1024**2:.0f} MB')
print(f'With optimisations : {t2:.2f}s,
      {df2.memory_usage(deep=True).sum()/1024**2:.0f} MB')
# Without: 3.21s, 87 MB
# With : 0.61s, 11 MB – 5× faster, 87% less memory
```

⚠ COMMON PITFALL / MISTAKE

Incorrect — mean of chunk means:

python – Code Example

```
means = []
for chunk in pd.read_csv('big.csv', chunksize=100_000):
    means.append(chunk['revenue'].mean())
print(np.mean(means)) # WRONG — last chunk may have fewer rows,
                      # pulling mean down
```

Correct — weighted mean via sum/count:

python — Code Example

```
total_sum, total_count = 0.0, 0
for chunk in pd.read_csv('big.csv', chunksize=100_000):
    total_sum += chunk['revenue'].sum()
    total_count += len(chunk)
print(total_sum / total_count) # Correct overall mean
```

INTERVIEW CARD

- **Frequency:** Very high for data engineering roles — chunking is a core production skill
- **Question:** How would you compute the total revenue per customer from a 5 GB CSV file on a machine with 8 GB RAM?
- **Expected Answer:** Use `pd.read_csv('file.csv', chunksize=100_000, usecols=['customer_id', 'revenue'], dtype={ ... })`. For each chunk, compute `chunk.groupby('customer_id')['revenue'].sum()` and append to a list. After all chunks, `pd.concat(partial_results).groupby('customer_id').sum()` produces the correct final result. Never load the full file; never compute mean from chunk means — track sum and count separately instead.
- **Follow-up:** Why can't you compute a correct median or percentile using chunked processing without additional tricks?

PRACTICE QUIZ

1. What does `pd.read_csv('file.csv', chunksize=100_000)` return?
2. Why must you do a second `groupby().sum()` after `pd.concat(partial_results)` ?
3. Is chunked median computation correct using the same approach as chunked sum? Why?
4. Predict: a 2M-row file read with `chunksize=100_000` — how many chunks are produced?
5. What does the `usecols` parameter do in `pd.read_csv` ?

6. Debug: chunked groupby gives a different total than full-file groupby — what is the most likely cause?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 **ANSWER KEY**

1. A `TextFileReader` iterator — lazy; no data is read until you iterate
2. Because the same `customer_id` can appear across multiple chunks — the per-chunk groupby produces partial sums that must be re-summed to get the correct total
3. No — median requires knowing the full sorted distribution. You cannot compute the true median from chunk medians. Approximate methods (T-Digest) exist but are not built into Pandas.
4. 20 full chunks of 100,000 rows + 1 final chunk with the remainder
5. Tells the CSV parser to read only the specified columns — reduces I/O, memory, and parse time proportionally
6. Forgetting the final re-aggregation step — the first groupby only aggregates within each chunk, not across chunks

❓ **CHECKPOINT QUESTION**

You need to compute the count of orders and average revenue per category from a 10 GB CSV. You cannot compute `mean` directly from chunk means. Design the complete chunked pipeline — what do you accumulate per chunk, and how do you compute the final mean correctly?

ELIMINATING APPLY(AXIS=1): VECTORISATION PATTERNS

ChatGPT • AI Assistant

A two-panel execution diagram. Left panel labeled 'apply(axis=1) — Python loop': shows a vertical stack of 1,000,000 row boxes. A Python interpreter icon sits at the top with a loop arrow going down through each row box sequentially. Each row has annotations showing overhead steps: 'unpack row → dict', 'call lambda function', 'type check', 'pack result'. A timer annotation at the bottom: '~8 seconds for 1M rows'. Right panel labeled 'Vectorised — C-level operations': shows the same 1,000,000 rows as a single compact block labeled 'NumPy contiguous memory'. A single C-level function arrow passes over the entire block at once, labeled 'df[a] * df[b] — one C function call'. A timer at the bottom: '~0.05 seconds — 160x faster'. Below both panels, a mapping table: 'row[a] * row[b] → df[a] * df[b] | if/else → np.where | multiple if/elif/else → np.select | row[col].upper() → df[col].str.upper()'. Purpose: make the execution model difference visceral and provide the substitution lookup table in the same visual.

apply(axis=1) vs Vectorised Operations: Execution Model Comparison

I use `.apply(lambda row: ..., axis=1)` a lot for row-wise operations. I've heard it's slow but what do I use instead?

ChatGPT • AI Assistant

CONCEPT CARD

`.apply(axis=1)` is the single worst-performing Pandas pattern. It iterates over every row in Python — one function call per row. For a 1M-row DataFrame, that is 1 million Python function calls, each with argument packing, type checking, and return value unpacking. The result is 10–200× slower than the vectorised equivalent.

The rule: **if you are writing ``apply(axis=1)``, there is almost always a vectorised alternative.** The alternatives depend on what the function does:

- **Arithmetic on columns** → use direct column arithmetic
- **Conditional assignment** → use `np.where` or `pd.Series.map`
- **String operations** → use `.str` accessor
- **Date operations** → use `.dt` accessor
- **Complex multi-column logic** → use `np.select` for multi-condition case statements

EXAMPLE CARD

Example 1 — Basic: Simple arithmetic

python – Code Example

```
import pandas as pd
import numpy as np
import time

# SLOW: apply for simple arithmetic
start = time.time()
df['total'] = df.apply(lambda row: row['quantity'] *
row['unit_price'], axis=1)
t_apply = time.time() - start

# FAST: vectorised column arithmetic
start = time.time()
df['total'] = df['quantity'] * df['unit_price']
```

```
t_vec = time.time() - start

print(f'apply : {t_apply:.3f}s')
print(f'vectorised: {t_vec:.4f}s')
print(f'Speedup: {t_apply / t_vec:.0f}x')
# apply : 8.234s
# vectorised: 0.047s
# Speedup: 175x
```

Example 2 — Intermediate: Conditional logic — np.where and np.select

python – Code Example

```
# SLOW: apply with if/else
df['size_label'] = df.apply(
    lambda row: 'Large' if row['quantity'] ≥ 5
                else ('Medium' if row['quantity'] ≥ 3 else 'Small'),
    axis=1
)

# FAST: np.select for multi-condition case statement
conditions = [
    df['quantity'] ≥ 5,
    df['quantity'] ≥ 3
]
choices = ['Large', 'Medium']
df['size_label'] = np.select(conditions, choices, default='Small')

# For simple two-way condition: np.where
df['is_high_value'] = np.where(df['revenue'] > 1000, 'High',
                              'Standard')
```

Student

What about when I need to call a custom Python function on each row? Some logic can't easily be vectorised.

ChatGPT • AI Assistant

First, try hard to vectorise — most logic that looks impossible to vectorise can be broken down into column operations. But when you genuinely need a custom function, there is a hierarchy of performance options:

1. **Fully vectorised** (best) — pure column arithmetic, NumPy ufuncs
2. **`.apply()` on a Series** (axis=0, the default) — operates column by column, not row by row. Acceptable for single-column transformations.
3. **Numba `@jit` or Cython** — compile your Python function to machine code. Near-C speed.
4. **`.apply(axis=1)`** (worst) — only use when no alternative exists

Example 3 — Practical: String operations — str accessor vs apply

python — Code Example

```
# SLOW: apply for string manipulation
df['category_upper'] = df.apply(lambda row: row['category'].upper(),
                                axis=1)

# FAST: str accessor — vectorised string operations
df['category_upper'] = df['category'].str.upper()
df['category_lower'] = df['category'].str.lower()
df['cat_len']        = df['category'].str.len()
df['starts_e']       = df['category'].str.startswith('E')
df['cat_cleaned']    = df['category'].str.strip().str.replace(' ',
    '_', regex=False)
```

Example 4 — Real Business: Complex multi-column discount calculation

python — Code Example

```
# Business rule: discount tier based on quantity AND category
# SLOW: apply
def compute_discount(row):
    if row['category'] == 'Electronics' and row['quantity'] ≥ 3:
        return 0.15
    elif row['quantity'] ≥ 5:
        return 0.10
    elif row['category'] == 'Clothing':
        return 0.05
    else:
        return 0.0

# FAST: np.select — vectorised multi-condition case
cond_elec_bulk = (df['category'] == 'Electronics') & (df['quantity']
    ≥ 3)
cond_bulk      = df['quantity'] ≥ 5
cond_clothing  = df['category'] == 'Clothing'
```

```
df['discount_rate'] = np.select(
    condlist=[cond_elec_bulk, cond_bulk, cond_clothing],
    choicelist=[0.15, 0.10, 0.05],
    default=0.0
)

df['final_price'] = df['revenue'] * (1 - df['discount_rate'])

print(df[['category', 'quantity', 'revenue', 'discount_rate', 'final_price']
        ].head(8))
```

Example 5 — Advanced: Benchmarking apply vs vectorised vs Numba

python – Code Example

```
import time

# Create test series for benchmarking
x = pd.Series(np.random.uniform(0, 1000, 1_000_000))

# Method 1: apply (Python loop)
start = time.time()
result1 = x.apply(lambda v: v ** 2 + 2 * v + 1)
t1 = time.time() - start

# Method 2: vectorised NumPy
start = time.time()
result2 = x ** 2 + 2 * x + 1
t2 = time.time() - start

# Method 3: NumPy ufunc directly
start = time.time()
arr = x.values # Get underlying NumPy array
result3 = arr ** 2 + 2 * arr + 1
t3 = time.time() - start

print(f'apply (Python loop): {t1:.3f}s')
print(f'Pandas vectorised : {t2:.4f}s - {t1/t2:.0f}x faster')
print(f'NumPy on .values : {t3:.4f}s - {t1/t3:.0f}x faster')
# apply: 0.824s
# Pandas: 0.009s - 91x faster
# NumPy: 0.006s - 137x faster
```

❗ COMMON PITFALL / MISTAKE

Incorrect — apply for column concatenation:

python – Code Example

```
df['full_label'] = df.apply(  
    lambda row: row['category'] + '_' + row['region'], axis=1  
) # Slow Python string concat per row
```

Correct — vectorised string concatenation:

python – Code Example

```
df['full_label'] = df['category'] + '_' + df['region'] # Uses Pandas  
str vectorisation  
# Or even faster with .str.cat():  
df['full_label'] = df['category'].str.cat(df['region'], sep='_')
```

 INTERVIEW CARD

- **Frequency:** Very high — this is asked in nearly every data engineering and ML engineering interview
- **Question:** Why is `.apply(axis=1)` slow and what are the vectorised alternatives?
- **Expected Answer:** `.apply(axis=1)` iterates over rows in Python — one function call per row with full interpreter overhead. For 1M rows that is 1M Python function calls. Alternatives: direct column arithmetic (`df.a * df.b`), `np.where` for two-way conditionals, `np.select` for multi-condition case statements, `.str` accessor for string operations, `.dt` accessor for date operations. Speed difference is typically 10–200×.
- **Follow-up:** When is `.apply(axis=1)` genuinely the right tool despite being slow?

 AI DEEP DIVE PROMPT

Copy this into Claude or ChatGPT:

"Act as a senior data engineer who cares deeply about Pandas performance. I want to practice replacing slow Pandas anti-patterns with fast vectorised alternatives. Give me 12 exercises in order of increasing complexity. Each exercise should:

1. Show me a slow Pandas code snippet using `apply(axis=1)`, a Python loop, or an inefficient pattern
2. Show the DataFrame schema and approximate row count
3. Ask me to rewrite it using vectorised operations
4. After I attempt it, show the optimal solution

5. Show the benchmarked speed difference using `timeit`
6. Explain WHY the vectorised version is faster at the memory/CPU level
7. Give one interview question about this pattern

Cover: arithmetic, conditional (`np.where`, `np.select`), string (`.str`), date (`.dt`), multi-column lookups (`pd.merge` vs `map`), and window functions. Do not continue until I attempt the rewrite."

EXERCISE

Rewrite these two `apply` patterns without using `.apply(axis=1)` :

1. ```python

```
df['profit_margin'] = df.apply(
    lambda row: (row['revenue'] - row['cost']) / row['revenue']
    if row['revenue'] > 0 else 0,
    axis=1
)
```

code — Code Example

```
2. ```python
df['label'] = df.apply(
    lambda row: 'Premium' if row['revenue'] > 2000 and
row['quantity'] > 3
                else ('Standard' if row['revenue'] > 500 else
'Budget'),
    axis=1
)
```

PRACTICE QUIZ

1. What is the primary reason `.apply(axis=1)` is slow?
2. What vectorised method replaces a two-condition if/else in `apply`?
3. What replaces `.apply(lambda row: row['name'].upper(), axis=1)` ?
4. Predict: which is faster — `df['a'].apply(np.sqrt)` or `np.sqrt(df['a'])` ?
5. What does `np.select(conditions, choices, default=0)` do when no condition matches a row?

6. Debug: `df['new'] = df[['a','b']].apply(lambda row: row.a * row.b, axis=1)` — rewrite this.

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 **ANSWER KEY**

1. It iterates in Python — one function call per row with full interpreter overhead (type checks, argument packing, reference counting). No C-level vectorisation.
2. `np.where(condition, value_if_true, value_if_false)`
3. `df['name'].str.upper()`
4. `np.sqrt(df['a'])` is faster — it calls the NumPy ufunc directly on the underlying array without the Series overhead of `.apply`
5. Returns the `default` value (0 in this case) for any row where no condition is True
6. `df['new'] = df['a'] * df['b']`

❓ **CHECKPOINT QUESTION**

You inherit a pipeline that runs `.apply(axis=1)` on a 5M-row DataFrame — it takes 45 seconds. The lambda function checks three conditions on two columns and returns one of four string labels. Write the vectorised replacement and estimate the new runtime.

WHEN PANDAS BREAKS DOWN: POLARS, DASK, AND PYSPARK

ChatGPT • AI Assistant

A diamond-shaped decision flowchart with four levels. Entry point at top labeled 'Your dataset'. First diamond: 'Data > 100 GB?' — Yes branch leads to 'PySpark (cluster required)' with annotation 'JVM overhead worth it at this scale'. No branch continues down. Second diamond: 'Data > 10 GB?' — Yes branch leads to 'Dask (out-of-core, Pandas API)' with annotation 'Larger than RAM, single machine'. No branch continues down. Third diamond: 'Building new pipeline AND no ML lib output?' — Yes branch leads to 'Polars (lazy, multi-threaded, Rust)' with annotation '1-10 GB + no legacy: Polars wins on speed'. No branch continues down. Fourth diamond: 'Data < 1 GB OR needs scikit-learn output?' — Yes leads to 'Pandas (+ dtype optimisations)' with annotation 'In-memory, ML-compatible, well-documented'. No leads to 'Pandas + chunking' with annotation 'Apply usecols, dtype, chunksize'. On the right side, a vertical bar shows relative performance: Pandas (baseline 1x), Pandas optimised (2-3x), Polars (5-10x), Dask multi-core (3-8x), PySpark cluster (10-100x for huge data). Purpose: give students a single reference for tool selection that they can return to when starting any new data project.

Tool Selection Decision Map: Pandas vs Polars vs Dask vs PySpark

I've heard of Polars and Dask as alternatives to Pandas. When should I actually switch? And does that mean I have to relearn everything?

ChatGPT • AI Assistant

CONCEPT CARD

Pandas is the right tool for most data work under ~5 GB that fits in memory. Beyond that, you hit Pandas' fundamental architectural limits:

1. **Single-threaded by default** — GroupBy, apply, and most operations use only one CPU core
2. **In-memory only** — if the data doesn't fit in RAM, Pandas fails
3. **No query optimiser** — every operation executes immediately, no reordering or pushdown

Three alternatives exist, each solving a different constraint:

- **Polars** — same single-machine in-memory model as Pandas but built in Rust, multi-threaded by default, with a query optimiser. 3–20× faster than Pandas on typical analytical queries.
- **Dask** — Pandas API for datasets larger than RAM, distributed across cores or a cluster. Each `dask.DataFrame` is a collection of Pandas DataFrames computed lazily.
- **PySpark** — distributed computing framework for truly massive datasets (100 GB to petabytes). Different API, JVM overhead, needs a cluster.

🗣️ ANALOGY

Pandas is a **sports car** — fast, comfortable, one driver. Dask is a **bus** — same roads, carries more passengers but harder to manoeuvre. PySpark is a **cargo train** — enormous capacity, dedicated tracks (cluster), completely different operating model.

EXAMPLE CARD

Example 1 — Basic: The same GroupBy in Pandas vs Polars

python — Code Example

```
# --- PANDAS (eager, single-threaded) ---
import pandas as pd
import time

start = time.time()
result_pd = (
```

```

df
    .query('revenue > 100')      # filter
    .groupby('category')        # group
    .agg(total_rev=('revenue', 'sum'), n_orders=('order_id', 'count'))
    .reset_index()
)
t_pd = time.time() - start

# --- POLARS (lazy, multi-threaded, query optimised) ---
import polars as pl

df_pl = pl.from_pandas(df) # Convert once

start = time.time()
result_pl = (
    df_pl
    .lazy()                    # Enter lazy mode
    .filter(pl.col('revenue') > 100) # filter
    .group_by('category')       # group
    .agg([
        pl.col('revenue').sum().alias('total_rev'),
        pl.col('order_id').count().alias('n_orders')
    ])
    .collect()                 # Execute the query plan
)
t_pl = time.time() - start

print(f'Pandas : {t_pd:.3f}s')
print(f'Polars : {t_pl:.3f}s')
print(f'Speedup: {t_pd/t_pl:.1f}x')
# Pandas : 0.231s
# Polars : 0.047s
# Speedup: 4.9x

```

Student

Why is Polars faster on the same machine with the same data?

ChatGPT • AI Assistant

Four reasons:

1. **Rust implementation** — zero garbage collector pauses, no GIL

2. **True multi-threading** — GroupBy and filter operations use all CPU cores automatically
3. **SIMD instructions** — Polars uses CPU vectorised instructions (AVX2/AVX-512) that process multiple values per CPU cycle
4. **Lazy query optimiser** — `.lazy()` builds a query plan; `.collect()` executes it. The optimiser can reorder operations (filter before join, push predicates down) to minimise work — Pandas executes every operation immediately with no lookahead

Example 2 — Intermediate: Dask for out-of-core processing

python – Code Example

```
import dask.dataframe as dd

# Read a large file as a Dask DataFrame (lazy – file not loaded yet)
ddf = dd.read_csv('/tmp/large_orders.csv',
                  dtype={'customer_id': 'int32', 'revenue':
                        'float32'})

print(type(ddf))      # <class 'dask.dataframe.core.DataFrame'>
print(ddf.npartitions) # Number of partitions (chunks)

# Dask operations look identical to Pandas – they build a task graph
result = (
    ddf[ddf['revenue'] > 100]
    .groupby('category')['revenue']
    .sum()
)
# Nothing has been computed yet – result is a Dask Series (lazy graph)

# .compute() triggers actual execution across all partitions
print(result.compute()) # NOW it runs
```

Student

When is Dask better than just using chunked Pandas?

ChatGPT • AI Assistant

Chunked Pandas requires you to manually write the aggregation logic per chunk and combine results. Dask automates this — you write standard Pandas-like code and Dask handles the chunking, parallelism, and result combination. Dask is better when:

- Your data exceeds RAM (Dask uses disk for spill-over)
- You want parallel execution across multiple CPU cores automatically
- Your pipeline has complex operations that are hard to decompose manually for chunking

Dask is NOT better when:

- Your data fits in memory — Pandas is faster (no task graph overhead)
- You need operations not supported by Dask's Pandas subset
- You are in a production environment where Polars would be faster (Dask parallelises Pandas, which is already slower than Polars)

Example 3 — Real Business: Decision framework in code

python – Code Example

```
def choose_data_tool(file_size_gb, is_new_pipeline, needs_ml_compat):
    """
    Returns the recommended tool and rationale.
    file_size_gb      : size of the dataset in GB
    is_new_pipeline   : True if building from scratch (no legacy code)
    needs_ml_compat   : True if scikit-learn / XGBoost etc. expected as
    output
    """
    if file_size_gb > 100:
        return 'PySpark', 'Dataset too large for single machine – need
cluster'

    if file_size_gb > 10:
        return 'Dask', 'Data too large for RAM – Dask handles out-of-
core'

    if file_size_gb > 1 and is_new_pipeline and not needs_ml_compat:
        return 'Polars', '1–10 GB + new pipeline: Polars wins on speed,
no legacy constraint'

    if needs_ml_compat:
        return 'Pandas', 'scikit-learn/XGBoost need Pandas/NumPy
arrays'

    if file_size_gb <= 1:
        return 'Pandas', 'Small data: Pandas is fastest to write and
debug'

    return 'Pandas + optimisations', 'Apply dtype downcast + chunking
first'
```

```
# Decision examples
for size, new, ml in [(0.5, True, True), (2, True, False), (2, True, True),
(15, True, False), (200, True, False)]:
    tool, reason = choose_data_tool(size, new, ml)
    print(f'{size:5.1f} GB | new={new} | ml={ml} → {tool}: {reason}')
```

❗ COMMON PITFALL / MISTAKE

Incorrect — using PySpark for 500 MB of data:

python – Code Example

```
# PySpark has JVM startup overhead of 15–30 seconds
# For 500 MB, Pandas processes in < 2 seconds
# Using Spark here makes a 2-second job into a 45-second job
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate() # 20 seconds of JVM
startup
df = spark.read.csv('500MB_file.csv')      # Then processing time ...
```

Correct — match tool to data size:

python – Code Example

```
# For 500 MB: Pandas with dtype optimisation
df = pd.read_csv('500MB_file.csv',
                 dtype={'customer_id': 'int32', 'revenue': 'float32'})
df = reduce_mem_usage(df) # Bring to ~200 MB
result = df.groupby('category')['revenue'].sum() # Fast in-memory
```

SUMMARY CARD

Section 5 Key Takeaways:

- `df.memory_usage(deep=True)` is the only accurate memory measurement — shallow measurement lies for object columns
- `int64` → `int16/int32` saves 50–75%; `float64` → `float32` saves 50%; `object` → `category` saves 50–98%
- `reduce_mem_usage()` automates dtype downcasting — apply it early in every pipeline
- **Do not use float32 for cumulative financial sums** — rounding errors compound
- `pd.read_csv(chunksize=N)` returns a lazy iterator — only one chunk in RAM at a time

- Always use `usecols` and `dtype` parameters with `pd.read_csv` for large files
- Chunked mean requires separate sum/count accumulators — mean of chunk means is wrong
- `.apply(axis=1)` is 10–200× slower than vectorised alternatives — eliminate it
- Replace apply with: column arithmetic, `np.where`, `np.select`, `.str`, `.dt`, `.map`
- Pandas for < 1 GB; Pandas + optimisations for 1–5 GB; Polars for 1–10 GB new pipelines; Dask for > 10 GB single machine; PySpark for > 100 GB cluster

INTERVIEW CARD

- **Frequency:** Very high — this entire section is standard senior data engineering interview territory
- **Question:** Walk me through how you would handle processing a 10 GB CSV file on a 16 GB RAM machine using Pandas.
- **Expected Answer:** (1) Use `pd.read_csv(usecols=..., dtype=..., chunksize=500_000)` to read only needed columns with correct types. (2) Apply dtype downcasting within each chunk. (3) Compute decomposable aggregations (sum, count) per chunk, track sum+count separately for mean. (4) Combine partial results with `pd.concat().groupby().sum()`. (5) If the pipeline logic is too complex for manual chunking, reach for Dask. If rebuilding the pipeline from scratch with no ML library constraint, consider Polars.
- **Follow-up:** What aggregations cannot be computed correctly via chunked processing?

PRACTICE QUIZ

1. What is the main architectural advantage of Polars over Pandas?
2. What does `.lazy()` do in a Polars pipeline?
3. What is the key difference between Dask and chunked Pandas?
4. Predict: reading a 10 GB file with `usecols=['a','b']` out of 50 columns — how much faster is this than reading all columns?
5. At what approximate data size does PySpark become worth its JVM overhead?
6. Debug: `ddf.groupby('cat')['rev'].sum()` prints without errors but shows no numbers — why?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. Multi-threading (no GIL) + lazy query optimiser + SIMD CPU instructions — all absent in Pandas
2. Switches Polars to lazy evaluation mode — operations build a query plan rather than executing immediately; `.collect()` triggers execution
3. Dask automates chunking, parallelism, and result combination with a Pandas-like API; chunked Pandas requires manual aggregation logic and combination per chunk
4. Approximately 50× less I/O and proportionally less parse time — $2/50 = 4\%$ of the original bytes read
5. Roughly > 100 GB or when a compute cluster is available — below this, Dask or Polars on a single machine is faster with less operational overhead
6. Dask operations are lazy — `ddf.groupby()['rev'].sum()` builds a task graph but doesn't execute; you must call `.compute()` to get results

🔍 CHECKPOINT QUESTION

You are building a new analytics pipeline for a 4 GB daily events file. The pipeline filters to active users, groups by user segment and event type, computes total event count and mean session duration, and outputs to a Parquet file. Your team uses scikit-learn downstream. Which tool do you choose and why? What specific optimisations do you apply?

🌟 CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including performance and chunking, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **Three Levers of Pandas Performance Optimisation:** A horizontal three-panel roadmap diagram. Panel 1 labeled 'LEVER 1 — Memory Redu...